# How Many Runs Does an RL Post-Training Comparison Need? Forecasting Run-to-Run Spread from a Single Run

Mehmet Demir Güven[†]
**ETH Zürich**[*]
dgueven@student.ethz.ch

**Abstract**

Reinforcement learning post-training is compared from single runs, and how many runs a comparison needs is set by a quantity nobody measures: how much a rerun would move the score. The usual recourse is to fix the run count in advance, or to spend runs estimating the spread before the comparison can begin, because after one run per arm there is no empirical spread to put in a stopping rule.

We forecast it instead. Training noise is not accumulated: a perturbation entering at one update is reweighted by every update that follows, so the covariance of a run about its mean obeys $S_{t+1} = A_t S_t A_t^\top + Q_t$. Carrying the gradient of the reported metric backwards along one stored trajectory turns that recursion into the across-seed variance of the metric, as a sum of scalars with no covariance matrix formed. On enumerable policies, where every input is exact, and frozen before the remaining seeds existed, it has a median error of 1.10× over 192 settings.

Used to size comparisons whose ordering is clear, with every stopping threshold chosen on other tasks before it is applied, it reaches a 0.1% mistake rate in 3.14 runs, 5.6 to 8.2× fewer than a fixed budget measured below its cap and 2.12× fewer than estimating the deviation from spent seeds. A closed form that needs no run comes within 1.11× once its threshold is learned on other tasks; the stored run's scale replaces that learning. Near-ties are settled by no policy within the seeds we have.

On pretrained models accuracy is regime-dependent. Across 4 pre-specified regimes on Qwen2.5-0.5B the frozen forecast misses by factors of $1.66, 2.13, 2.80, 1.24$, inside the 4.2× relative error at which, with $z$ set in sample, the procedure stops paying. On a task where most runs learn to answer one constant it is short by a factor of 25, and at 7B under Adam by 3.1 to 17×. We report both.

## 1 Introduction

Reinforcement learning with verifiable rewards is the standard way to turn a pretrained model into a reasoning one [DeepSeek-AI et al., 2025], and a paper reports that a recipe moved a benchmark by two points. Whether that is a result depends on a number nobody has: how much the same recipe would have moved it under a different seed. The problem is documented [Henderson et al., 2017, Hochlehnert et al., 2025], and the standard instrument for it is replication, which at RL post-training cost is close to unaffordable.

The practical form of the problem is not reporting but budgeting. Deciding that one recipe beats another means training seeds of each until the lead is convincing, and how many that takes is governed by the across-seed deviation. A practitioner who does not have it either fixes the seed count in advance, or spends seeds estimating it before the comparison can start. Both are expensive, and the second is expensive for a structural reason: after a single run per arm there is no empirical deviation at all, so the first two runs of every arm buy no evidence about the question being asked.

This paper forecasts that deviation from one run, and then spends it. The route runs through a correction to the obvious model of where run-to-run spread comes from. That model says gradient noise perturbs each update, perturbations accumulate, two runs execute a random walk away from each other, and reproducibility decays with run length. It is wrong. Noise from one update does not survive to the end of training unchanged: it is passed through every update that follows, which reweights it. What reaches the end is a convolution of injected noise with the sensitivity of the remaining trajectory, an object that obeys a recursion and that can be carried against the gradient of the reported metric rather than being formed.

**Contributions.**

- **An error bar from one run** (Theorem 2). Carrying the metric gradient backwards along a single stored trajectory gives the across-seed variance of the reported score as a sum of scalars, one per update, with no covariance matrix formed. The

[*]Department of Computer Science, ETH Zürich, 8092 Zurich, Switzerland.

[†]ETH Zürich neither endorsed, supervised nor funded this work; the author was an undergraduate in its BSc Computer Science programme during this research, hence the affiliation.

transpose a backward pass would need is free, because the mean update of any count-based estimator is the gradient of a scalar potential and its Jacobian is therefore symmetric. On enumerable policies, where every input to the forecast is exact, and frozen before the remaining seeds were trained, its median error is 1.10× across 192 settings of at most 25 updates.

- **Comparisons sized from one run** (Section 6). With every stopping threshold chosen on other tasks before it is applied, the forecast settles 1020 comparisons at a 0.1% mistake rate in 3.14 runs, against 14 for an uncapped fixed budget and 2.12× as many for a deviation estimated from spent seeds. A closed form that needs no run does nearly as well once its threshold is learned (1.11× [0.97, 1.28]); what the stored run adds is an absolute scale, which lets the textbook threshold be used with no calibration at all.
- **The condition under which it pays.** Corrupting the forecast deliberately puts the break-even against a fixed budget at 4.2× relative error; a pure scale error is free only where the threshold can be tuned on comparisons with known outcomes.
- **An honest boundary** (Section 7). On Qwen2.5-0.5B accuracy is regime-dependent: across 4 pre-specified regimes the errors are $1.66, 2.13, 2.80, 1.24$, all inside the break-even above. Outside it, on a task where most runs learn to answer one constant whatever they are asked, the forecast is short by 25×, and at 7B under Adam the forecast is short by between 3.1 and 17.

Every experiment runs on one laptop, and every pretrained forecast spans at most 12 updates with LoRA adapters, so none of it establishes behaviour at frontier scale (Section 9).

## 2 Setup

A policy $\pi_\theta(y \mid x)$ answers a prompt $x \sim \mathcal{D}$ and a verifier returns a binary reward $r(x, y) \in \{0, 1\}$. The objective is $J(\theta) = \mathbb{E}_x[p(x)]$ with $p(x) := \Pr[r = 1 \mid x]$. Write $\boldsymbol{s}(y) := \nabla_\theta \log \pi_\theta(y \mid x)$, so $\mathbb{E}_y[\boldsymbol{s}] = 0$, and let $u_r := \mathbb{E}[\boldsymbol{s} \mid r]$. Because the reward is binary, every quantity below depends on the policy only through the per-prompt objects $(p, u_1, S_0, S_1)$, where $S_r := \mathrm{Cov}(\boldsymbol{s} \mid r)$, which is what makes an exact finite-$G$ treatment possible.

A batch is $P$ prompts drawn from $\mathcal{D}$ with $G$ responses each; $R = PG$ is fixed by hardware. With $k$ the number of successes in a group we consider any advantage of the form

$$A_j = w(k, r_j), \tag{1}$$

that is, any rule in which a response's advantage depends only on its own reward and on how many of its group-mates succeeded. It covers RLOO [Ahmadian et al., 2024], GRPO with a mean baseline [Liu et al., 2025], standardised GRPO [Shao et al., 2024] and a fixed external baseline. The per-prompt and batch estimators are $z := G^{-1} \sum_j w(k, r_j) \boldsymbol{s}(y_j)$ and $\hat{\boldsymbol{g}} := P^{-1} \sum_i z_i$.

**Two levels of noise.** Prompts are sampled and then responses are sampled within each prompt, so by the law of total variance

$$\mathrm{Cov}(\hat{\boldsymbol{g}}) \;=\; \frac{1}{P}\Big[\, \Sigma_{\mathrm{b}} + \tfrac{1}{G-1}\Sigma_{\mathrm{w}} \,\Big], \tag{2}$$

with $\Sigma_{\mathrm{b}}$ the covariance across prompts of the per-prompt mean update and $\Sigma_{\mathrm{w}}$ the within-prompt term, written in the $(G-1)$ convention the leave-one-out estimators use. Both traces are recoverable from the microbatch gradients a trainer already accumulates. We write $\tau_{\mathrm{b}}$ and $\tau_{\mathrm{w}}$ for those traces.

**Intervals.** Seed experiments produce pairwise quantities: $n$ runs give $\binom{n}{2}$ distances, which are not $\binom{n}{2}$ observations, since every run appears in $n-1$ of them. Treating them as independent understates the standard error by roughly a factor of two at the seed counts anyone trains. Every interval here resamples *runs* and rebuilds the pairwise quantity from the resampled set.

## 3 Noise is filtered, not accumulated

Fix a policy $\theta_0$ and start two runs from it. They share parameters, optimiser state, data distribution and recipe, and independently draw everything the seed controls: which prompts each batch contains and which responses are sampled. That is the comparison a practitioner makes when they rerun, and it is the estimand throughout, so both terms of (2) contribute.

Write $\delta_t := \theta_t - \bar{\theta}_t$ for one run's departure from the mean trajectory. With updates $\theta \leftarrow \theta + \eta \hat{\boldsymbol{g}}$ and $\mathcal{J}_t := \partial \bar{\boldsymbol{g}} / \partial \theta$,

$$\delta_{t+1} = A_t \delta_t + \eta\, \xi_t, \qquad A_t := I + \eta \mathcal{J}_t, \tag{3}$$

to first order in $\delta$, where $\mathbb{E}[\xi_t \mid \mathcal{F}_t] = 0$ and $\mathrm{Cov}(\xi_t) = \Sigma_t / P$.

If the objective were flat, $\mathcal{J} = 0$, this would be a random walk: divergence would accumulate as $\mathbb{E}\|\Delta_t\|^2 \propto t$ and a longer run would be strictly less reproducible.

That is the implicit model behind measuring run-to-run variance by brute force. But two RLVR runs climb the same objective, and $A_t$ reweights a perturbation rather than passing it through.

**Theorem 1** (Finite-time covariance)**.** *Let* $S_t := \mathrm{Cov}(\delta_t)$*, with* $S_0 = 0$ *when runs start together. Under* (3)*,*

$$S_{t+1} = A_t S_t A_t^\top + Q_t, \qquad Q_t := \eta^2 \Sigma_t / P, \tag{4}$$

*hence* $S_T = \sum_{t<T} \Phi_{T,t+1} Q_t \Phi_{T,t+1}^\top$ *with* $\Phi_{T,s} := \prod_{u=s}^{T-1} A_u$ *taken in order of decreasing* $u$*. Two independent seeds satisfy* $\mathrm{Cov}(\theta_T^A - \theta_T^B) = 2S_T$*.*

*Proof sketch.* Taking the covariance of (3) leaves a cross term $\mathbb{E}[\delta_t \xi_t^\top]$. It vanishes without assuming $\xi_t$ and $\delta_t$ are independent, which they are not, since the batch noise is drawn at $\theta_t$ and that parameter was produced by earlier noise. What holds is that $\xi_t$ is a martingale difference, $\mathbb{E}[\xi_t \mid \mathcal{F}_t] = 0$, and $\delta_t$ is $\mathcal{F}_t$-measurable, so the cross term is zero by the tower rule. Unrolling from $S_0 = 0$ gives the sum. □

The factor of two belongs to the difference of two runs and not to the covariance of one; keeping them apart is what makes the statement unambiguous.

### 3.1 Carrying it to the number that gets reported

Equation (4) concerns policies. What a paper reports is a score $M(\theta)$: pass rate, exact match, win rate. Linearising about the endpoint, $\mathrm{Var}_{\mathrm{seed}}[M(\theta_T)] \approx \nabla M^\top S_T \nabla M$, which needs a matrix nobody can form for a real model. It is not needed.

**Theorem 2** (Adjoint form of the reported spread)**.** *Define* $b_T := \nabla M(\theta_T)$ *and* $b_s := A_s^\top b_{s+1}$ *for* $s = T-1, \ldots, 0$*. Then*

$$V_T^{lin} = \sum_{t=0}^{T-1} b_{t+1}^\top Q_t \, b_{t+1} = \frac{\eta^2}{P} \sum_{t=0}^{T-1} \mathrm{Var}_i\big(b_{t+1}^\top z_{t,i}\big), \tag{5}$$

*the variance taken over prompts.*

*Proof sketch.* $b_s = \Phi_{T,s}^\top \nabla M$ by induction; substitute the unrolled $S_T$. The second equality is $Q_t = \eta^2 \Sigma_t / P$ with $b^\top \Sigma_t b = \mathrm{Var}_i(b^\top z_{t,i})$, since $\Sigma_t$ is the covariance of one prompt's contribution rather than of the batch mean. □

The equality is exact for $V_T^{\mathrm{lin}}$, the variance of the metric linearised at $\bar{\theta}_T$. The quantity a practitioner wants is $\mathrm{Var}_{\mathrm{seed}}[M(\theta_T)]$, and the two agree to the order at which the linearisation holds; Section 7 is about when that agreement fails. Every term of (5) is one scalar per update, which a trainer can accumulate alongside the update it is already making.

**Algorithm 1** The forecast, as a trainer would run it

```
train as usual, storing θ_t and η_t for each update
b ← ∇M(θ_T)   ▷ one rollout and backward pass on the eval set
v ← 0
for t = T − 1 down to 0 do
    draw a batch at θ_t; let ĝ_{t,i} be its per-prompt gradients
    v ← v + (η_t^2 / P) · Var_i(b^⊤ ĝ_{t,i})        ▷ one scalar
    if transport is needed then
        δ ← ε b / ‖b‖
        b ← b + (η_t / 2) · (‖b‖ / ε) (ḡ(θ_t + δ) − ḡ(θ_t − δ))   ▷ common random numbers
    end if
end for
return √v
```

What the backward pass costs depends on $A_s^\top$, and here the estimator family pays for itself.

**Proposition 1** (The mean update is a gradient field)**.** *For any count-based estimator,* $\bar{\boldsymbol{g}}(\theta) = \nabla_\theta \Psi(\theta)$ *with* $\Psi(\theta) = \mathbb{E}_x[\Lambda(p_x)]$ *and* $\Lambda' = \lambda$*, where the per-prompt mean update is* $\lambda(p_x, G)\nabla_\theta p_x$*. Hence* $\mathcal{J} = \nabla^2 \Psi$ *is symmetric and* $A_s^\top = A_s$*.*

Because $\lambda$ depends on $\theta$ only through $p_x$, $\lambda(p_x)\nabla p_x = \nabla \Lambda(p_x)$, and averaging over $x$ commutes with the gradient. We call $\Psi$ the *estimator potential*; it is not the objective $J$ unless $\lambda \equiv 1$, and the gap is exactly the difficulty reweighting the estimator applies. For RLOO the potential is the pass rate itself. The practical consequence is that the backward pass needs no transpose and can be run with the same code path as the forward one.

## 4 Running it

**What it costs, plainly.** As written, the backward pass makes three passes per stored update: one batch for the injected term and two for the Hessian-vector product. That is about three times the compute of the training run, not a rounding error, and we would rather state it than let a reader discover it.

It is also usually unnecessary. Ablating the transport, holding $b$ at $\nabla M(\theta_T)$ instead of carrying it back, changes the forecast by a median of 2% and the median error not at all over runs of ten and twenty-five updates. The reason is in the spectrum of $A_t$: most of it sits within $10^{-4}$ of one and $\nabla M$ points into that part, so $\Phi_{T,t}$ is close to the identity along the one direction the forecast needs. What we recommend is the

cheap estimator,

$$\mathrm{Var}_{\mathrm{seed}}\left[M(\theta_T)\right] \;\approx\; \sum_t \frac{\eta_t^2}{P}\,\widehat{\mathrm{Var}}_i\big(\nabla M(\theta_T)^\top \hat{g}_{t,i}\big), \quad (6)$$

one extra pass per update and no Hessian-vector products, with the full recursion held in reserve for runs long enough, or curvature strong enough, that a per-update discrepancy of a percent compounds. Knowing which regime one is in is what the recursion is for.

**A closed form with no run at all.** Take (6) with $\Phi \approx I$ and a constant step, replace the projection $\mathrm{Var}_i(\nabla M^\top \hat{g}_{t,i})$ by the trace (2), and the run drops out entirely:

$$\sigma \;\propto\; \eta\sqrt{T/P}\,\sqrt{\tau_{\mathrm{b}} + \tau_{\mathrm{w}}/(G-1)}. \quad (7)$$

We call this the *frozen* forecast because it needs only the run length, the batch shape and the two noise traces, and is therefore available before any training happens.

It is written as a proportionality on purpose. The constant it is missing carries $\|\nabla M\|$, which is exactly the part that requires a trained model, and on our grid (7) overstates the measured deviation by a factor of about 84. That factor is not quite constant: it varies by 4.8× across the grid, leaving a relative error of 1.26× at the median once one common scale is removed. So (7) is useless for reporting an error bar and, by the argument of Section 6, entirely serviceable for sizing one, because a stopping threshold absorbs a constant and nothing absorbs the part that varies.

Both traces come from the microbatch gradients a trainer already accumulates: with blocks $a$ holding disjoint prompts and sub-groups $b \in \{1,2\}$ independent given a prompt,

$$\begin{aligned} \operatorname{tr}\Sigma_{\mathrm{b}} &= \tfrac{P}{K}\big(\mathbb{E}[g_{a1}^\top g_{a2}] - \mathbb{E}[g_{a1}^\top g_{a'2}]_{a\neq a'}\big), \\ \operatorname{tr}\Sigma_{\mathrm{w}} &= \tfrac{P}{2K}\,\mathbb{E}\big[\|g_{a1} - g_{a2}\|^2\big]. \end{aligned} \quad (8)$$

The cost is $2K$ gradient buffers rather than one, no extra responses and no extra backward work. Estimating the total and subtracting the within-prompt part instead differences two large correlated quantities and is badly conditioned: on homogeneous pools it returns a negative $\tau_{\mathrm{b}}$. Against exactly known values over pools whose noise ratio spans a factor of 26, (8) recovers the optimal group size to a median error of 1.3%, while the subtractive form reports 6.1 where the truth is 9.4.

Section 6 finds that (7) does most of the work, which matters because it costs almost nothing.

# 5 Does the forecast work

We test where the answer can be checked exactly. The environments are small enough to enumerate every sequence, so $\bar{\boldsymbol{g}}$, the Fisher and the metric gradient are computed in closed form rather than sampled, and the forecast is compared against a spread measured from 47 independent seeds. The grid spans two policy shapes, two prompt-pool diversities, three difficulty bands, two group sizes and a range of step sizes: 192 settings in all, over which the measured spread itself varies by 46×.

| grouped by | settings | median | worst | inside 95% |
|---|---|---|---|---|
| all settings | 192 | 1.10× | 1.56× | 162/192 |
| policy $3^3$ | 96 | 1.10× | 1.56× | 82/96 |
| policy $4^2$ | 96 | 1.10× | 1.38× | 80/96 |
| $T = 10$ | 96 | 1.10× | 1.34× | 86/96 |
| $T = 25$ | 96 | 1.11× | 1.56× | 76/96 |
| $P = 8$ | 96 | 1.12× | 1.56× | 78/96 |
| $P = 32$ | 96 | 1.08× | 1.53× | 84/96 |
| $\eta = 0.5$ | 96 | 1.10× | 1.29× | 92/96 |
| $\eta = 1.5$ | 96 | 1.13× | 1.56× | 70/96 |

**Table 1:** Forecast against measured spread, prospectively. Error is the ratio in whichever direction exceeds one.

The test is prospective. One run is trained, the forecast is computed from it alone, and only then are the remaining seeds trained. The anchor is excluded from the measured spread, since a predictor that is also one of its own targets is not independent of them.

Table 1 reports a median error of 1.10× and a worst case of 1.56×, with the forecast inside the measured 95% interval in 162 of 192 settings. The grid uses a pool of 24 prompts; the same 192 settings on a pool of 16, run as a replicate, give 1.10×, 1.50× and 154 of 192.

**The recursion is what predicts divergence.** Two cheaper stories are available for the full covariance, and both fail. Accumulation, the random-walk picture that ignores $A_t$ entirely, is wrong by a factor of two on homogeneous prompt pools, which is where the objective's curvature is most consistent and the filtering therefore strongest; on heterogeneous pools it is wrong by three percent, which is what (4) predicts. A single effective timescale fails by two orders of magnitude. Measuring the spectrum of $A_t$ says why neither works: it is not a contraction. The spectral radius exceeds one throughout, by up to 2% early and decaying as the policy improves, and only 31–51% of directions contract. Divergence stays bounded because expansion is slow relative to run length, not because every direction shrinks.

**The filtering is caused by learning.** Replacing rewards with coin flips, which leaves the rollouts, the batch composition and the step sizes untouched, turns a divergence trace that ends at 0.14 of where it started into one that grows by a factor of 90.5 over eighty updates, a gap of 671× at the end. The learning con-

dition also moves further per update, so what brings those runs back together is not smaller steps.

# 6 What the forecast is for

A predicted standard deviation is not a result on its own. Nobody trains a model in order to learn how much its score would move under a different seed; they want to know whether one recipe beats another, and the deviation is what says how many runs settle it.

## 6.1 The decision

Two configurations are to be ordered. A practitioner trains each for a seed, looks at the lead, and either calls it or trains another seed of each. Stopping when the lead $\bar{D}_k$ after $k$ seeds per arm clears $z$ standard errors,

$$|\bar{D}_k| \;\geq\; z\,\frac{\sigma_D}{\sqrt{k}}, \qquad \sigma_D = \sqrt{\sigma_A^2 + \sigma_B^2}, \tag{9}$$

is the rule this instantiates. It needs $\sigma_D$, and that is the difficulty: at $k = 1$ there is no empirical deviation to put there. The remaining choices are to fix $k$ in advance, which is what is normally done, or to spend seeds building a variance estimate before the comparison can begin. Sections 3 and 4 supply $\sigma_A$ and $\sigma_B$ from the first run of each arm, so (9) is available from $k = 1$.

Multiplying every $\sigma$ by a common factor is undone by dividing $z$ by it, so if $z$ can be tuned, only the *relative* deviation across configurations matters. Tuning $z$ needs comparisons whose outcome is known; Section 6.4 asks what remains without them.

## 6.2 Setup

The scores are the 48 per-seed scores already stored for each of the 192 configurations of Section 5; no new training is done, and every policy sees the same seeds in the same order, so the comparison is paired. We compare configurations sharing a task.

The true ordering has to come from somewhere, and reading it from seeds a policy is also shown would correlate the policy's evidence with the answer it is graded against, flattering whichever policy looks at the most seeds. So 32 of the 48 seeds are reserved: the ordering is fixed on those and no policy may draw one. The 1020 pairs whose reserved-seed ordering is unambiguous at four standard errors are kept, so a disagreement is a mistake rather than a coin landing the other way up, and policies draw from the remaining 16. Each pair is replayed under 400 random orderings.

A policy is a curve rather than a point, since $z$ trades runs against mistakes, and reading every curve at a common mistake rate is what makes two comparable.

| policy | runs | vs. adjoint | 95% |
|---|---|---|---|
| fixed budget, no stopping rule | 30.41 | 9.57 | [7.24, 10.56] |
| deviation from the seeds in hand | 6.52 | 2.13 | [1.98, 2.38] |
| one deviation for every recipe | 6.04 | 1.93 | [1.45, 2.28] |
| frozen forecast, before any run | 3.34 | 1.09 | [1.02, 1.14] |
| adjoint forecast, from one run | 3.07 | 1.00 | [1.00, 1.00] |
| measured deviation | 3.16 | 1.02 | [0.99, 1.05] |

**Table 2:** Runs needed to hold the mistake rate at 0.1% over 1020 comparisons, both arms counted, with 95% intervals from resampling the comparisons. The forecast is not better than knowing the deviation; it is not measurably worse, which is the point.

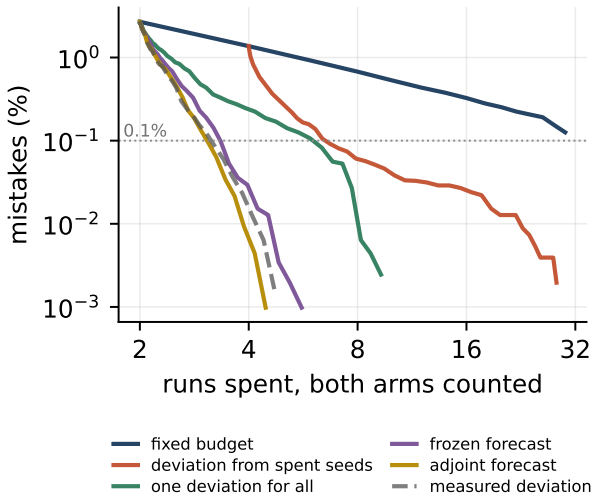



**Figure 1:** Mistakes against runs spent. A policy is a curve, not a point, since the stopping threshold trades one against the other, and what separates two policies is how far left a curve sits at a fixed height. At 0.1% the forecasts cross near 3.07 runs and a fixed budget near 30.4. The rule driven by spent seeds flattens on the left because it cannot stop before two seeds per arm however large the lead.

We report the runs each needs to hold its mistake rate at 0.1%, both arms counted.

## 6.3 Result, with $z$ read in sample

The adjoint forecast reaches 3.07 runs and a rule handed the realised deviation 3.16, the same to within resampling error ($1.02\times$ $[0.99, 1.05]$). A fixed budget needs 30.4 runs, $9.57\times$ $[7.24, 10.56]$ more, and estimating the deviation from spent seeds $2.13\times$ more, mostly because that rule cannot stop at one seed per arm. An earlier version graded against all 48 seeds, a policy's own included; that leak favours the baselines that draw the most seeds and cut the fixed-budget gap to $4.76\times$. The closed form (7), with no stored run and no backward pass, reaches 3.34 runs, within $1.09\times$ of the adjoint: most of the saving does not need Theorem 2.

| | target 1% | | target 0.1% | |
|---|---|---|---|---|
| threshold set on other tasks | runs | mistakes | runs | mistakes |
| fixed budget | 6.00 | 0.91% | 32.00 | 0.00% |
| one deviation for every recipe | 2.47 | 0.98% | 6.24 | 0.12% |
| deviation from spent seeds | 4.05 | 1.02% | 6.66 | 0.10% |
| closed form, no run | 2.30 | 0.96% | 3.48 | 0.08% |
| closed form, one scale from other tasks | 2.29 | 0.98% | 3.47 | 0.08% |
| adjoint forecast | 2.23 | 0.98% | 3.14 | 0.10% |
| measured deviation | 2.22 | 0.95% | 3.22 | 0.09% |
| textbook threshold, no tuning | | | | |
| one deviation for every recipe | 4.36 | 0.22% | 5.77 | 0.11% |
| closed form, no run | 32.00 | 0.00% | 32.00 | 0.00% |
| closed form, one scale from other tasks | 4.64 | 0.01% | 6.23 | 0.00% |
| adjoint forecast | 4.37 | 0.00% | 5.96 | 0.00% |
| fixed budget, 24-seed cap, 3 splits | 4.0–4.0 | | 15.3–23.3 | |

**Table 3:** Runs per comparison and realised mistake rate over 1020 comparisons with no threshold read off the comparisons graded. Upper block: threshold fitted on the other eleven tasks. Lower block: the textbook threshold. At 0.1% the fixed budget sits at the 16-seed cap; the last row extrapolates it without one.

### 6.4 Setting the threshold before the comparison

Those numbers read every policy at the threshold meeting the target on the comparisons it is graded on: even-handed, but not something a practitioner can do. Table 3 repeats the benchmark two ways one could. Either the threshold, or for a fixed budget the seed count, is the cheapest meeting the target on the other eleven tasks and is applied unchanged to the twelfth; or there is no tuning, and the rule stops once the lead clears the one-sided normal quantile of the target. Intervals resample the twelve tasks, since comparisons inside a task share arms.

Thresholds chosen on other tasks transfer: realised mistake rates land near the target. At 0.1% the adjoint forecast needs 3.14 runs; the fixed budget sits at its cap here, so we also split the seeds 24/24, where it can be measured below a cap: over 3 splits it needs 15.3 to 23.3 runs, 5.6 to 8.2× the forecast. Against the closed form it is 1.11× [0.97, 1.28] with exact noise traces and 1.18× with traces estimated from one run: once $z$ can be learned on other tasks, the stored run adds little to a forecast that only has to rank.

At the textbook threshold the uncalibrated closed form never stops, its scale off by 84×; given one scale learned on other tasks it decides in 4.64 runs with 0.013% mistakes at a 1% target, and the adjoint forecast in 4.37 with 0.001%. The stored run's scale stands in for that one learned factor, and both textbook rules spend about twice the tuned runs. Keeping near-ties (two standard errors instead of four) puts every policy at the cap at 0.1%; no policy settles near-ties with the seeds we have.

The other use of a deviation is choosing: run several configurations once and keep the one with the most gain per unit of noise. With seeds split three ways, never reused across roles, the choice keeps 66% of the best achievable value with one deviation for every recipe and 75% with the forecast, against 77% with the measured deviation: the forecast recovers 8.8 points [1.7, 15.6] of 10.2. Choosing has little headroom because one pilot's gain is itself noisy.

### 6.5 How wrong the forecast may be

On a pretrained model the forecast is accurate only in some regimes (Section 7), so we corrupt it on purpose and rerun the benchmark: a lognormal multiplicative error of controlled median magnitude, and separately a systematic scale error.

The scale error does nothing here. Over 100-fold changes in scale the runs needed are identical to the digits we print, a factor of 1.000, because $z$ absorbs it exactly as (9) says. That holds only because this benchmark reads $z$ off the graded comparisons; with the textbook threshold a scale error is not absorbed, which is why the closed form never stops in Table 3. Calibration is free only to someone who already has comparisons with known outcomes to set $z$ on.

Relative error is what costs, and it costs less than we expected. At the 0.1% target the runs needed rise from 3.07 at the measured accuracy to 5.4 at a median error of 1.5×, 9.5 at 2×, 13.3 at 2.5× and 18.7 at 4×, and never reach the fixed budget's 30.4 anywhere in the range we corrupted, which extends to 12×. At a looser 1% target, where a fixed budget needs only 5.61 runs, the break-even is 4.2×.

That break-even, measured with $z$ read in sample, is the statement of scope. The 4 pre-specified regimes of Section 7, at 1.66, 2.13, 2.80, 1.24, sit inside it. Two of our pretrained results do not: the task on which most runs learn to answer one constant, missed by 25×, and Qwen2.5-7B under Adam on the same task, short by between 3.1 and 17, a range reaching well past it. Where the forecast misses that badly, a forecast-driven stopping rule would stop too early.

## 7 On pretrained models

Run end to end on Qwen2.5-0.5B with LoRA, the cheap estimator (6) is a factor of 25 short of the measured spread and the transported one 16. On Qwen2.5-7B under Adam, with the recursion lifted to the optimiser state $z = (\theta, m, v)$, it is short by between 3.1 and 17. On the exact models the lift gives 1.10× with Adam's moments warmed, but started cold and without bias correction, as these runs were, its median error there is 2.40× and its worst 549×.

**Accuracy is regime-dependent, not uniformly poor.** On the corpus above the reward groups go unanimous within five updates, yet its measured seed

spread is the largest we report. Most runs there learn to answer one constant: the stored run says "1" to every prompt, and which constant a run falls into explains 94% of the spread. So we built four corpora at pre-specified mean live-group probabilities and ran the prospective test on each. The errors are $1.66, 2.13, 2.80, 1.24$; the best, on the band at 0.85, meets a criterion of $1.5\times$ declared before the sweep ran, forecasting 0.0205 against a measured 0.0253 $[0.0105, 0.0337]$ on a gain of 0.275. So local covariance forecasting can be quantitatively accurate on a pretrained model, though not reliably so.

All four sit inside the $4.2\times$ break-even of Section 6, which was measured with $z$ read in sample. That is the narrow sense in which these results are usable: in those four regimes the forecast is accurate enough for the decision it serves, while on the unanimous corpus and at 7B under Adam it is not. Running the sizing benchmark itself at this scale is the direct test, and it is underpowered.

**The sizing benchmark at this scale.** We trained 6 recipes on one corpus at a matched rollout budget of 64 per update, crossing $(P, G) \in \{(16, 4), (8, 8)\}$ with three learning rates spanning a factor of four: 102 training runs. The forecast holds up, at a median error of $1.61\times$ and a worst of $2.75\times$, all six inside the break-even, tracking $3.0\times$ of the $4.7\times$ spread in measured deviation across recipes.

The benchmark does not. Only 4 of 15 pairs are ordered unambiguously at 2 standard errors, because the recipes differ in mean by at most 0.025 while their deviations run from 0.010 to 0.048: most are genuine near-ties at sixteen seeds, which is the paper's premise appearing in its own data. On four comparisons a stopping rule beats a fixed budget by 1.41 $[1.28, 1.65]$ and nothing else separates, the ordering among the $\sigma$-aware policies changing with the seed cap. That the measured deviation itself comes out nominally behind the forecast, at $1.05\times$, marks the resolution floor rather than a result. The $9.57\times$ of Section 6 is therefore not demonstrated at this scale and we do not claim it.

**Two things the sweep falsified.** It does not support a monotone relationship between stochastic support and accuracy: the ordering of errors across the four bands does not follow the ordering of their live-group probabilities. And a one-run curvature diagnostic, calibrated on the exact models to refuse forecasts whose runs are not small perturbations of each other, does not transfer: it refuses every pretrained run, including the accurate one.

**What survives is narrower than the magnitude.** The share of prompts whose group is not unanimous falls from about a half to under 5% by the fifth update, which is measured rather than modelled, so the run stops injecting seed noise there. The failed forecast's own kernel puts 90% of its predicted variance before that point; whether the actual seed variance is distributed the same way we did not test.

**A metric can go blind before the seeds agree.** At 7B on a task the base model already mostly passes, twenty updates collapse the response distribution: every held-out prompt scores exactly 0 or 1, two seeds report the same number to the last decimal, and their policies still differ by up to $8 \times 10^{-2}$ in KL. A zero measured spread on a saturated benchmark is not evidence of a reproducible procedure, and it is the failure mode that a practitioner is most likely to read as success.

# 8 Related work

**Run-to-run variance.** That RL results move under reseeding is old news in deep RL [Henderson et al., 2017] and documented again for language-model reasoning [Hochlehnert et al., 2025]. Those papers establish the size of the problem by replication, and Colas et al. [2018] turn it into a power analysis that sizes the seed count from a pilot sample of seeds, which is the practice our sizing experiment replaces. Agarwal et al. [2021] prescribe how the resulting samples should be summarised. All of it presumes the runs exist. Heineman et al. [2025] estimate a benchmark's noise within one run, from checkpoint to checkpoint; ours is the across-seed spread of the training procedure. Practice for benchmark error bars treats the evaluation set as the only source of randomness [Miller, 2024], which is the right interval for a fixed model and the wrong one for a training procedure; Bouthillier et al. [2021] argue the pipeline's own randomness belongs in the interval and estimate it by replication. We estimate the same quantity without replicating, and our real-model measurements subtract the binomial term explicitly.

**Deciding when to stop repeating.** Stopping a comparison once one arm is convincingly ahead is sequential analysis [Wald, 1945], and its machine-learning form is racing [Maron and Moore, 1993, Even-Dar et al., 2006], sharpened by using an empirical variance in the bound [Audibert et al., 2009] and scaled to hyperparameter search by allocating a budget across candidates [Li et al., 2017]. Section 6 uses the simplest rule of that family deliberately: the contribution is not a better stopping rule but the quantity the rule consumes. Every method there takes its variance from samples already drawn, which is sound when a sample is a validation example and unaffordable when a sample is a training run. At one run per arm those

bounds are vacuous, and the experiment shows that the resulting floor, not the shape of the bound, is what a comparison pays.

**Stochastic dynamics of training.** Treating stochastic optimisation as a process whose covariance is set by gradient noise against curvature is standard [Mandt et al., 2017], and (4) is not new as mathematics. What we add is the RLVR-specific measurement of $Q_t$ through the two-level decomposition, the finite-time rather than stationary treatment, and the observation that the filter, not the size of the noise, is what governs reproducibility here. Wang et al. [2026] reach a filtering picture from the opposite side, reporting near-linear evolution of weights and log-probabilities; their object is one run's trajectory, ours the covariance between runs differing only in seed. Closest to us, Ghosh et al. [2026] reduce GRPO to a scalar damped oscillator with a stationary fluctuation scaling as $1/G$; we keep the full covariance, at finite time, and test it on seeds trained after the forecast was frozen.

**Adjoint sensitivity.** Theorem 2 applies the adjoint method [Pontryagin et al., 1962, Chen et al., 2018] to a covariance rather than a loss.

**Entropy collapse.** That RLVR drives policy entropy down sharply is documented [Cui et al., 2025] and treated as an optimisation pathology. It has a second consequence that follows from (5): a count-based advantage is identically zero on a unanimous group, so a collapsed policy injects $\Sigma_t = 0$, stops accumulating seed uncertainty, and any measurement of its reproducibility from that point is measuring nothing.

**Batch size and gradient noise.** The gradient noise scale predicts the batch size beyond which data parallelism stops buying speed [McCandlish et al., 2018, Shallue et al., 2018]. Both concern a single level of sampling. RLVR samples prompts and then responses within each prompt, and the resulting two-level scale has no counterpart there; the decomposition itself is the classical two-stage sampling result [Cochran, 1977]. Zhou et al. [2026] concurrently derive an optimal group size from the mean-squared error of the GRPO gradient as a U-statistic; ours minimises injected noise at fixed rollout budget, and the two remain uncompared.

**Allocating rollouts.** Recent methods vary responses per prompt within a batch from pass rates or gradient variance [Nguyen et al., 2026], and outside language models the batch size itself has been adapted during training [Park, 2026]. Those allocate samples *within* one run. The budget at issue here is across independent training runs of the same recipe, which is a level those methods do not address and where the reproducibility cost is actually incurred.

# 9 Limitations

**The sizing benchmark.** The stopping rule (9) is not anytime-valid. Testing after every seed and stopping on the first crossing inflates the error of a fixed-sample test, so its nominal $z$ carries no coverage guarantee and we claim none. What we report is the mistake rate the rule realises, measured against an ordering fixed on seeds no policy may draw. Every policy is graded the same way and separated at a common realised rate, so the comparison does not rest on the boundary being calibrated; substituting a sequentially valid boundary would move all of these policies together, and we would expect it to change the constants and not the ordering. We have not verified that. Pairs whose ordering is ambiguous are excluded, so the benchmark says nothing about near-ties, where the honest answer is to keep training whatever rule is in use. And the decision is over one scalar metric.

**Scale.** The largest model is a 4-bit 7B with 10 seeds, on one laptop, and the exact tier where the forecast is validated is the furthest from a frontier run.

**What the forecast costs, which is the objection we cannot fully answer.** Estimating the injected term from the eight prompts a run trains on gives two forecasts 681× apart, because only about a tenth of groups are not unanimous and eight prompts rarely contain one. Raising that to 32 prompts brings the disagreement between disjoint halves to 5.5× and costs 141 minutes against the 260 that training all 10 seeds costs. A diagnostic consuming half the experiment it replaces is not yet a diagnostic at this scale. What limits it is how well a sparse sample pins down a variance to which most prompts contribute exactly zero; an estimator that spends its rollouts finding the live groups is the obvious next thing to build, and we have not built it. The closed form (7) avoids the backward pass, and on the exact tier estimating its traces from one run costs little, but on a pretrained model they face the same sparse sample, untested.

**Claims we withdrew.** A predicted square-root relation between policy divergence and outcome spread is not supported by our data. Nor is a short memory horizon: in 254 of 288 settings more of the final variance was injected in the first half of the run. Token-level advantage reshaping, length normalisation and clipping fall outside the estimator family (1) and are not covered.

**How this work was done.** The author chose the research questions, decided which experiments to run, judged the results, decided which claims to keep, correct or withdraw, and derived and checked parts of the theory. AI coding agents (Claude Code) wrote most of the code and much of the text, ran the experiment sweeps under that direction, and in a later revision found several of the errors this version corrects. Every number in the tables and in the generated macros is computed from the stored run records released with the code.

## References


Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

Jean-Yves Audibert, Rémi Munos, and Csaba Szepesvári. Exploration-exploitation tradeoff using variance estimates in multi-armed bandits. *Theoretical Computer Science*, 410(19):1876–1902, 2009.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.

William G. Cochran. *Sampling Techniques*. John Wiley and Sons, 3rd edition, 1977.

Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments. *arXiv preprint arXiv:1806.08295*, 2018.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025.

Eyal Even-Dar, Shie Mannor, and Yishay Mansour. Action elimination and stopping conditions for the

multi-armed bandit and reinforcement learning problems. *Journal of Machine Learning Research*, 7: 1079–1105, 2006.

Rajat Ghosh, Datta Nimmaturi, Aryan Singhal, Vaishnavi Bhargava, Henry Wong, Johnu George, and Debojyoti Dutta. Predictable GRPO: A Closed-Form Model of Training Dynamics. *arXiv preprint arXiv:2606.30789*, 2026.

David Heineman, Valentin Hofmann, Ian Magnusson, Yuling Gu, Noah A. Smith, Hannaneh Hajishirzi, Kyle Lo, and Jesse Dodge. Signal and Noise: A Framework for Reducing Uncertainty in Language Model Evaluation. *arXiv preprint arXiv:2508.13144*, 2025.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep Reinforcement Learning that Matters. *arXiv preprint arXiv:1709.06560*, 2017.

Andreas Hochlehnert, Hardik Bhatnagar, Vishaal Udandarao, Samuel Albanie, Ameya Prabhu, and Matthias Bethge. A Sober Look at Progress in Language Model Reasoning: Pitfalls and Paths to Reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2017.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-Like Training: A Critical Perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Stephan Mandt, Matthew D. Hoffman, and David M. Blei. Stochastic Gradient Descent as Approximate Bayesian Inference. *arXiv preprint arXiv:1704.04289*, 2017.

Oded Maron and Andrew W. Moore. Hoeffding races: Accelerating model selection search for classification and function approximation. In *Advances in Neural Information Processing Systems*, 1993.

Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An Empirical Model of Large-Batch Training. *arXiv preprint arXiv:1812.06162*, 2018.

Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.

Hieu Trung Nguyen, Bao Nguyen, Wenao Ma, Yuzhi Zhao, Ruifeng She, and Viet Anh Nguyen. Adaptive Rollout Allocation for Online Reinforcement Learning with Verifiable Rewards. *arXiv preprint arXiv:2602.01601*, 2026.

Jongchan Park. Scalable Reinforcement Learning via Adaptive Batch Scaling. *arXiv preprint arXiv:2605.21557*, 2026.

L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko. *The Mathematical Theory of Optimal Processes*. Interscience Publishers, 1962.

Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. Measuring the Effects of Data Parallelism on Neural Network Training. *arXiv preprint arXiv:1811.03600*, 2018.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2): 117–186, 1945.

Tianle Wang, Jiayu Liu, Zhongyuan Wu, Shenghao Jin, Wei Chen, Hao Xu, and Ning Miao. Linear Dynamics in the RLVR Training of Large Language Models. *arXiv preprint arXiv:2601.04537*, 2026.

Hongyi Zhou, Kai Ye, Erhan Xu, Jin Zhu, Ying Yang, Shijin Gong, and Chengchun Shi. Demystifying Group Relative Policy Optimization: Its Policy Gradient is a U-Statistic. *arXiv preprint arXiv:2603.01162*, 2026.